OPERATING SYSTEM (CT 656)

Chapter 1: Introduction

Tribhuvan University | BCT | Year III / Part II

Full Marks: 80 | Pass Marks: 32 | Time: 3 hrs.

Q1. What is an Operating System? Explain the functions of Operating System. [1+4]

Answer:

An Operating System (OS) is a system software that acts as an interface between the user and the computer hardware. It manages hardware resources and provides a platform for application programs to run. Without an OS, a computer is merely a dead machine — it cannot accept user input or perform any useful work.

Definition: An Operating System is a program that controls the execution of application programs and acts as an interface between the user of a computer and the computer hardware.

Examples: UNIX, Linux, MS-Windows, macOS, Android, iOS

Functions of an Operating System:

- Process Management: The OS creates, schedules, and terminates processes. It manages the CPU using scheduling algorithms (FCFS, SJF, RR, etc.) to ensure efficient utilization.
- Memory Management: The OS allocates and deallocates memory to processes. It handles virtual memory, paging, and segmentation to allow efficient memory use.
- File System Management: The OS manages files and directories. It handles creation, deletion, reading, writing, and access control of files.
- I/O Management: The OS controls all input/output devices through device drivers, providing a uniform interface to applications.
- Security and Protection: The OS protects system resources from unauthorized access using authentication, authorization, and access control mechanisms.
- Network Management: The OS manages network connections, protocols, and data transfer between systems.
- Error Detection and Handling: The OS continuously monitors the system for errors in hardware and software and takes corrective action.
- User Interface: The OS provides a command-line interface (CLI) or graphical user interface (GUI) for users to interact with the system.

Q2. Define Operating System. Justify the statement "Operating system acts as resource manager as well as virtual machine". [1+4]

Answer:

An Operating System is a software that manages computer hardware and software resources and provides common services for computer programs.

OS as a Resource Manager:

A computer system has many resources — CPU, memory, I/O devices, and files. Multiple users and programs compete for these resources simultaneously. The OS acts as a resource manager by:

- Allocating CPU time to processes using scheduling algorithms
- Allocating memory space to processes using memory management techniques
- Managing I/O devices, ensuring no two processes access the same device simultaneously
- Controlling access to files through file system management
- Resolving conflicts when multiple processes request the same resource

OS as a Virtual Machine:

A virtual machine is an abstraction of the underlying hardware that presents a simpler, more convenient interface to users. The OS acts as a virtual machine by:

- Hiding the complexity of hardware from users and programmers
- Providing system calls as an abstract interface to hardware services
- Making every user feel as if they have a dedicated machine with unlimited resources
- Providing a uniform interface regardless of the physical hardware differences
- Example: A programmer writing in Python does not need to know the details of the CPU instruction set or memory addresses — the OS handles all that.

Q3. Write down the major differences between the following types of Operating System: a) Batch System b) Interactive System c) Real Time System d) Time Sharing System [2+8]

Answer:

Type of OS	Key Characteristics
Batch System	Jobs are collected, grouped, and processed without user interaction. No immediate response. High throughput. E.g., payroll processing, bank statements.
Interactive System	User interacts directly with the system. Provides immediate response. Uses keyboard/mouse for input. E.g., desktop computing, web browsing.
Real Time System	Guarantees response within strict time deadlines. Used in critical applications. Hard RTOS (failure is catastrophic) and Soft RTOS (deadline miss is acceptable). E.g., aircraft control, pacemakers.
Time Sharing System	Multiple users share CPU time in turns (time slices/quantum). Each user gets the illusion of a dedicated machine. Response time is minimized. E.g., UNIX multi-user systems.
Multiprogramming System	Multiple programs reside in memory simultaneously. CPU switches between them when one waits for I/O, maximizing CPU utilization.

Type of OS	Key Characteristics
Multiprocessing System	Multiple CPUs in one system. Processes run in parallel. Increases throughput and reliability.
Distributed System	Multiple independent computers networked together, appearing as one system to the user. Resource sharing across the network.

Comparison Table:

Feature	Batch	Interactive	Real Time	Time Sharing
User Interaction	None	Continuous	Limited	Interactive
Response Time	Hours/Days	Immediate	Within deadline	Seconds
Scheduling	Simple FCFS	Priority-based	EDF/RM	Round Robin
CPU Sharing	One job at a time	Yes	Dedicated	Time slices
Example	Payroll	Desktop OS	Flight control	UNIX

Q4. Explain why exo-kernel doesn't require re-mapping of resources? Is layered structure of operating system better than monolithic structure? Explain. [2+3]

Answer:

Exo-Kernel and Re-mapping of Resources:

An exo-kernel is a minimalist OS design where the kernel exports hardware resources directly to applications without any abstraction. Traditional OS kernels create an abstract virtual machine (e.g., virtual memory, virtual files) which requires re-mapping: converting virtual addresses to physical addresses, virtual file offsets to disk blocks, etc.

Exo-kernel does NOT require re-mapping because:

- It exports physical resources (actual disk blocks, actual memory frames) directly to applications
- Applications manage their own resources using hardware-level primitives
- There is no translation layer — applications access the real hardware directly
- Library Operating Systems (LibOSes) running above the exo-kernel handle any required resource management themselves
- This eliminates the overhead of maintaining virtual-to-physical mappings in the kernel

Layered Structure vs. Monolithic Structure:

Layered Structure	Monolithic Structure
OS divided into layers, each built on the layer below	All OS functions combined into one large program in kernel space

Layered Structure	Monolithic Structure
Each layer provides services to the layer above it	All services (memory, file system, drivers) run in kernel mode
Easy to debug and test — each layer tested independently	Hard to debug — a bug anywhere can crash the entire OS
More modular and maintainable	Less modular, tightly coupled components
Can add/remove layers without affecting others	Changes to one component may affect others unpredictably
Performance overhead due to inter-layer calls	Better performance — direct function calls, no overhead
Example: THE OS by Dijkstra	Example: Unix, MS-DOS (earlier versions)

Conclusion: Yes, layered structure is generally better than monolithic for maintainability, debugging, and modularity. However, monolithic systems may have better performance since there are no inter-layer call overheads. Modern OS (like Linux) use a hybrid approach combining benefits of both.

Q5. What is the purpose of a system call in an operating system? Explain. [1+3]

Answer:

A system call is a programmatic way in which a computer program requests a service from the kernel of the operating system. It provides an interface between a process and the operating system, allowing user programs to access OS services.

Purpose of System Calls:

- To provide controlled access to hardware resources (CPU, memory, I/O) from user space
- To allow user programs to request OS services without knowing hardware details
- To maintain security — user programs cannot directly access hardware, only through system calls
- To provide a uniform interface regardless of the underlying hardware

Types of System Calls:

1. Process Control: fork(), exec(), exit(), wait(), getpid() — used to create, control, and terminate processes
2. File Management: open(), close(), read(), write(), stat() — used to manipulate files
3. Device Management: ioctl(), read(), write() — used to communicate with devices
4. Information Maintenance: getpid(), alarm(), sleep() — used to get/set system information
5. Communication: pipe(), shmget(), msgget(), socket() — used for inter-process communication

How System Call Works:

- User program calls a library function (e.g., printf)
- Library function prepares parameters and triggers a software interrupt (trap)

- CPU switches from user mode to kernel mode
- OS kernel handles the request and performs the required operation
- Result is returned to user program and CPU switches back to user mode

Q6. What do you mean by Shell? Discuss about Microkernel and Monolithic structuring with their advantages and disadvantages. [4]

Answer:

Shell:

A shell is a command-line interpreter that provides an interface between the user and the OS kernel. It reads commands from the user, interprets them, and passes them to the kernel for execution.

Examples: Bash, Zsh, C shell, Korn shell.

Monolithic Structure:

In a monolithic kernel, the entire OS runs as a single program in kernel mode. All OS services (memory management, file system, device drivers, process management) are included in one large kernel.

- Advantages: High performance (direct function calls), simple design, efficient communication between components
- Disadvantages: Hard to maintain and debug, a bug in one component can crash the entire system, not easily extensible
- Example: Traditional UNIX, Linux kernel

Microkernel Structure:

In a microkernel, only the most essential services (IPC, basic scheduling, basic memory management) run in kernel space. All other services (file system, device drivers, networking) run as user-space processes (servers).

- Advantages: More reliable and secure (a crashed server does not crash the kernel), easier to port to new hardware, easier to extend
- Disadvantages: Performance overhead due to frequent context switches between user-space servers and kernel, more complex design
- Example: Mach, MINIX, QNX, L4

Monolithic Kernel	Microkernel
All services in kernel space	Only core services in kernel space
Fast performance	Slower due to IPC overhead
Hard to maintain	Easy to maintain and extend
Less secure (one component failure = crash)	More reliable (server failures isolated)
Example: Linux, Unix	Example: Mach, MINIX, QNX

Q7. How multiprogramming helps to increase performance of the system? Explain system call with example. [5]

Answer:

Multiprogramming refers to the technique of keeping multiple programs in memory at the same time, allowing the CPU to switch between them. When one process waits for an I/O operation, the CPU is immediately given to another process.

How Multiprogramming Increases Performance:

- CPU Utilization: Without multiprogramming, the CPU sits idle whenever a process waits for I/O. With multiprogramming, the CPU is always busy executing another process.
- Throughput: More jobs are completed per unit of time because the CPU never sits idle unnecessarily.
- Memory Efficiency: Multiple programs share memory, and memory is used more effectively.
- Example: While Process A waits for disk I/O, Process B uses the CPU. When A's I/O completes, the scheduler gives A the CPU again. This way, both processes progress and CPU is rarely idle.

System Call Example — fork():

fork() is a system call that creates a new process (child process) as a copy of the calling process (parent process).

Example in C:

```
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) { /* child process */ }
    else { /* parent process */ }
```

Q8. What is an Operating system? Explain the statement "operating system acts as a Broker between hardware and application program". [1+4]

Answer:

An Operating System is a set of programs that manage computer hardware resources and provide common services for application software. It is the most important software that runs on a computer.

OS as a Broker between Hardware and Application Programs:

A broker is an intermediary that facilitates interaction between two parties. The OS acts as a broker between hardware and application programs in the following ways:

- Abstraction: The OS hides the complexity of hardware from application programs. Apps do not need to know the exact hardware model — they use standard OS APIs (system calls).
- Resource Allocation: Applications request resources (CPU, memory, files) from the OS, which allocates them from the hardware pool. Applications never directly access hardware.
- Translation: The OS translates high-level application requests (e.g., 'write to file') into low-level hardware commands (e.g., 'write sectors 234-237 on disk').

- Protection: The OS ensures that one application cannot accidentally or maliciously access hardware resources belonging to another application.
- Example: When a user program calls printf() to print text, the OS receives this request, translates it into appropriate I/O device commands, and sends the output to the monitor.

Diagram (textual): Application Program → System Call → OS (Broker) → Hardware

---

Q9. Explain spooling with an example. How does the operating system act as a central manager for critical resources like CPU, memory and storage? [3+4]

Answer:

Spooling (Simultaneous Peripheral Operations Online):

Spooling is a process in which data is temporarily held in a memory buffer (spool) to be used and executed by a device, program, or system when it is ready. It allows multiple programs to output data to a slow device (like a printer) without waiting.

Example — Printer Spooling:

- When multiple users send print jobs simultaneously, the OS doesn't send them all to the printer at once (the printer handles one job at a time)
- Instead, the OS stores all print jobs in a spool (temporary storage on disk)
- The printer daemon reads jobs from the spool one by one and sends them to the printer
- Users can continue working without waiting for the printer to finish
- This allows CPU and applications to be decoupled from slow peripheral devices

OS as Central Manager of Critical Resources:

CPU Management: The OS uses scheduling algorithms (Round Robin, SJF, Priority) to fairly and efficiently allocate CPU time to multiple processes. It maintains the ready queue and decides which process runs next.

Memory Management: The OS tracks which parts of memory are in use and by whom. It allocates memory to processes when needed and reclaims it when freed. Virtual memory techniques allow processes to use more memory than physically available.

Storage Management: The OS manages the file system — organizing data on disk, maintaining directories, controlling access permissions, and optimizing disk space usage. It uses disk scheduling (SSTF, SCAN, C-SCAN) to efficiently serve I/O requests.

I/O Device Management: The OS manages all I/O devices through device drivers, providing a uniform interface. It uses buffering, caching, and spooling to improve I/O performance.

Q10. Without operating system, a digital machine is called a dead machine. Why? Briefly describe about different structures of operating system. [5]

Answer:

Without an OS, a digital machine is called a dead machine because:

- No program can execute — there is no software to load and run user programs

- Hardware resources cannot be managed — CPU, memory, and I/O devices would be inaccessible
- No user interface — users cannot give commands or see outputs
- No file system — data cannot be stored, retrieved, or organized
- Security is absent — no access control or protection mechanisms
- Without the OS, the hardware is just electronic circuits with no intelligence or usability

Different Structures of Operating System:

1. Simple/Monolithic Structure: All OS functions in one layer. The entire OS is one large program running in kernel mode. Fast but hard to maintain. Example: MS-DOS.

2. Layered Structure: OS divided into layers (Layer 0 = hardware, Layer N = user interface). Each layer provides services to the layer above. Easy to debug but may have performance overhead. Example: THE OS.

3. Microkernel Structure: Only essential functions (IPC, scheduling, memory) in kernel. All other services as user-space processes. Reliable and extensible but slower. Example: Mach, MINIX.

4. Modular Structure (Loadable Kernel Modules): Core kernel with loadable modules for additional services (device drivers, file systems). Used in modern Linux — combine best of monolithic and microkernel.

5. Virtual Machine Structure: Hardware virtualized so multiple OS instances can run on one machine. Each VM thinks it owns the hardware. Example: VMware, VirtualBox, JVM.

6. Exo-Kernel Structure: Exports raw hardware resources to applications. Applications manage their own resources. Maximizes application-level optimization.

7. Client-Server Structure: Similar to microkernel. OS services implemented as servers. Client processes request services via messages. Used in distributed systems.